

TideWatch 技术文档（初稿）

面向多河道断面的河道水位与水质智能监测、预警与多端协同系统。本文档描述 D1–D5、E1–E3 阶段的实现设计，与 `contract.md`、`README.md` 及 `Evidence/` 中的证据一一对应。所有机制描述均以仓库实际代码为准。

1. 项目与需求

1.1 项目目标

TideWatch 面向一条河道的三个监测断面 reach-a（上游）、reach-b（中游）、reach-c（下游），持续采集水位（waterLevel）、浊度（turbidity）与水流等级（flowLevel），并完成以下闭环：

- **判级**：按统一规则把每个断面的实时数据判为洪水预警 / 高水位警戒 / 水质浑浊 / 正常（contract 第 3 节）；
- **预警**：异常数据自动创建事件，按 OPEN → HANDLING → RECOVERED 状态机流转（contract 第 6 节）；
- **优先关注**：用环境分 + 感知分 + 持续分计算每个断面的优先分，自动标出当前最值得关注的断面（contract 第 7 节）；
- **多端协同**：Web 监测台、移动端现场巡查、3D 河道场景围绕同一条 MQTT 数据流工作，事件与干预动作跨端同步（contract 第 10 节）；
- **对照裁决**：固定规则与 IsolationForest 异常检测结果对照，加权组合裁决，规则正常而 ML 明显异常时提示人工复核（contract 第 8 节）。

1.2 使用场景

角色	场景	对应端
值班人员	值班室大屏总览三断面实时状态、事件流转、规则/ML 对照	web/index.html
现场巡查员	手机上只看当前优先关注断面，一键执行干预动作	mobile/index.html
调度/展示	3D 河道上定位异常断面、观察水位柱与水流动画	map3d/index.html
复盘分析	历史 CSV 的统计与趋势报告	python/analyze.py → report/report.html
免手/取证	语音指令、结论朗读、现场拍照并生成感知记录	Web 多模态交互区（E2）

1.3 核心问题与设计约束

本项目要解决的核心问题不是“画一个监测大屏”，而是**多端围绕同一条数据流的正确性**：

1. 三个断面的数据不能串线，所有状态以 reachId 为唯一 key（contract 第 1 节）；
2. status 必须由规则计算，不能手填（contract 第 3 节）；
3. 事件状态转移必须走合法路径，重复、乱序、迟到消息不得破坏状态机（contract 6.2/6.3/6.5）；
4. 三端不能各自维护互不关联的状态，事件与干预必须跨端广播（contract 第 5 节、E3）；
5. ML 结果必须如实标注来源，simulated 数据不得伪造置信度（contract 4.3）。

1.4 与 R18 的关系

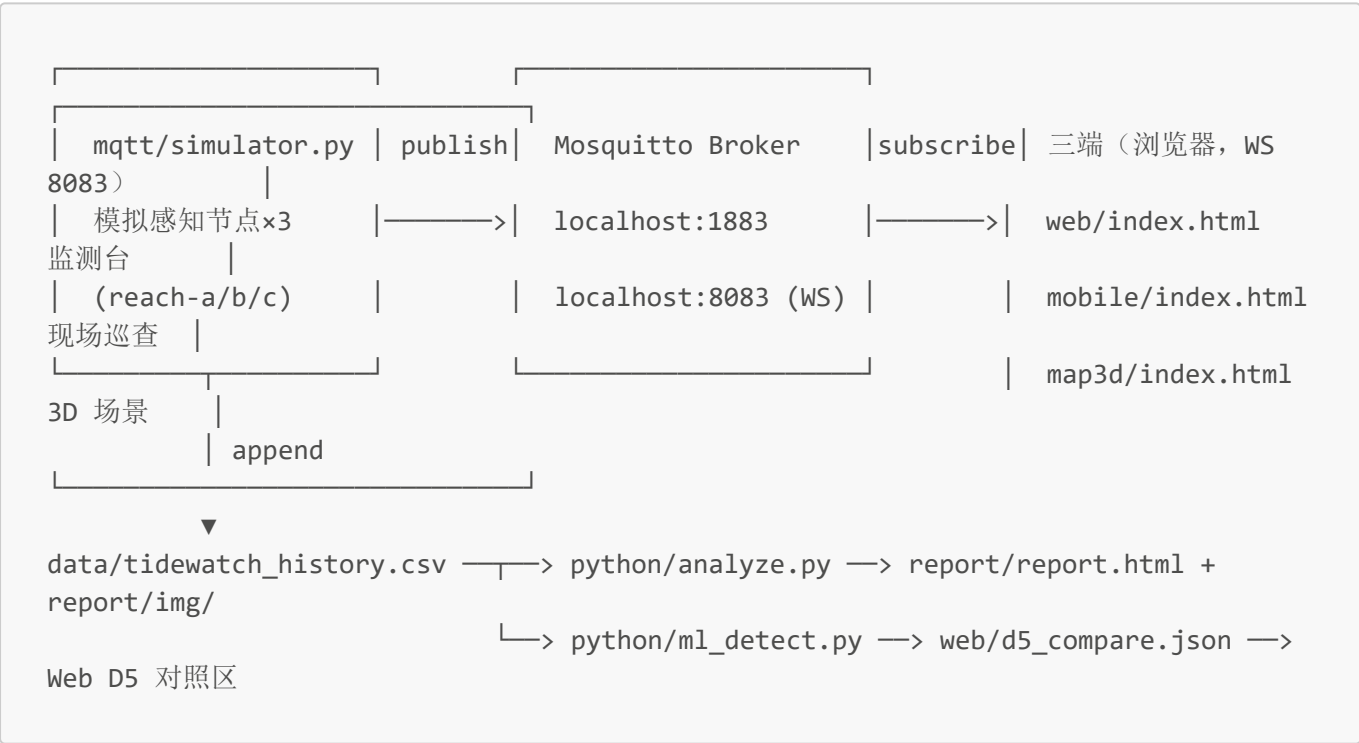
TideWatch 是 R18 挑战阶段后的综合作品，挑战阶段的六个模块为本项目提供了能力基础（证据索引见 Evidence/Challenge/README.md，证据保留在原仓库 nova-dormmate-final-2026）：

- M1 Web 主应用 / M5 MQTT/JSON 实时监测台 → **复用**：浏览器 + mqtt.js 的连接、订阅、渲染模式直接平移为 TideWatch 的 Web 监测台；
- M2 Python 离线分析 → **复用**：pandas + matplotlib 的分析链重组为 contract 9.1 的离线链（analyze.py → report.html）；
- M3 Camera/ASR/TTS → **复用**：三项本机交互能力平移为 E2 多模态区；
- M4 微信小程序 → **重新设计**：小程序改为一页 HTML 移动端（mobile/index.html），理由是与 Web 共享同一套 mqtt.js 技术栈、无需小程序账号与审核；
- M6 Three.js/3D → **复用**：3D 能力从"融合展示"升级为 E1 的实时数据驱动河道场景。

重新设计的是挑战阶段没有的东西：统一判级规则、事件状态机、优先关注评分、消息健壮性（去重/乱序/迟到）、规则/ML 对照——这些是 TideWatch 任务书的核心增量，也是本技术文档第 3、4 节的主体。

2. 系统架构与数据链

2.1 整体架构



- Broker 双监听：1883（MQTT）供模拟节点，8083（WebSocket）供浏览器三端，QoS 1，匿名访问（mqtt/mosquitto.conf）；
- 前端库全部本地化（mqtt.min.js、three.min.js r128、OrbitControls.js），不依赖 CDN 与外网——这是本项目早期诊断出 unpkg CDN 加载失败后的修复决策；
- Web 是事件状态的权威端：事件创建与状态变化由 Web 广播 `tidewatch/event/update`；干预动作两端都可发起，经 `tidewatch/intervention` 互相同步（E3）。

2.2 离线分析链（contract 9.1）

断面采集/历史记录 → CSV → Python → 时空统计/趋势 → report.html，实现如下：

环节	实现
历史记录	simulator.py 每次发布的同时把同一份 payload 追加到 data/tidewatch_history.csv（表头只写一次）；也可由 MQTT 模拟节点手动准备
统计	python/analyze.py 用 pandas 按断面统计：条数、水位均值/最大值、浊度均值/最大值、异常条数（status ≠ 正常）
趋势	matplotlib 画各断面水位/浊度趋势图，标注 8.5 / 7.0 警戒线，输出 report/img/*.png
报告	汇总写入 report/report.html，报告由脚本生成，不靠手工修改

2.3 边缘实时链（contract 9.2）

水文采集节点 → MQTT/JSON → 共享实时状态 → Web监测台 + 移动端 + 地图/3D

```

simulator.py                                tidewatch/{reachId}/state
(3 断面 × 3 轮, 每 3 秒一条) —————> web/index.html    订阅
tidewatch/+/state                           |
                                           |> mobile/index.html 订阅同一套 Topic
                                           |> map3d/index.html  订阅同一套 Topic

```

三端消费同一条消息：同一份 JSON（reachId, waterLevel, turbidity, flowLevel, status, time），各自按分工渲染。事件链路为：Web 创建/流转事件 → 广播 `tidewatch/event/update` → 移动端与 3D 更新；任一端干预 → 广播 `tidewatch/intervention` → 对端更新。移动端不发 event/update，避免广播环路。

3. 核心数据与系统机制

3.1 数据结构（contract 第 2 节）

统一 JSON 六字段：`reachId`, `waterLevel`, `turbidity`, `flowLevel`, `status`, `time`。time 格式 `YYYY-MM-DD HH:mm:ss`；status 只能由判级规则计算；flowLevel 取值 0–3（缓流/平稳/湍急/湍流），本阶段来源为 simulated（contract 4.3）。

事件对象字段（contract 6.6）：`event_id`, `reachId`, `start_time`, `problem`, `priority_reason`, `action`, `verify_data`, `state`, `recover_time`。event_id 由 Web 生成（`evt-{reachId}-{时间戳}`）；problem 记录触发原因（水位异常/水质异常/水流异常，可多选）；verify_data 记录 HANDLING 期间收到的后续数据。

感知结果记录字段（contract 4.2）：`reachId`, `imageId`, `flowLevel`, `confidence`, `source`, `time`。拍照后生成；source=simulated 时 confidence 恒为 null，不伪造置信度。

设计权衡：为什么恰好六个字段——身份（reachId）、两个环境量（waterLevel/turbidity）、一个感知量（flowLevel）、结论（status）、时间（time）构成监测链路的最小闭环：三端、事件、优先关注、ML 训练全部只需这六者，字段再多会放大三端各自实现的不一致风险。status 虽可由规则重算，仍随消息冗余携带，是为了让订阅端拿到“同一时刻的同一结论”，避免各端重算版本漂移。time 用本地时间而非 UTC：本系统是单机/局域网演示，无跨时区协同需求；`YYYY-MM-DD HH:mm:ss` 固定格式的字典序即时间序，字符串直接比较即可做乱序判断，免去各端解析与转时区的复杂度。

3.2 Topic 结构与消息处理管线

Topic 三张（contract 第 5 节）：

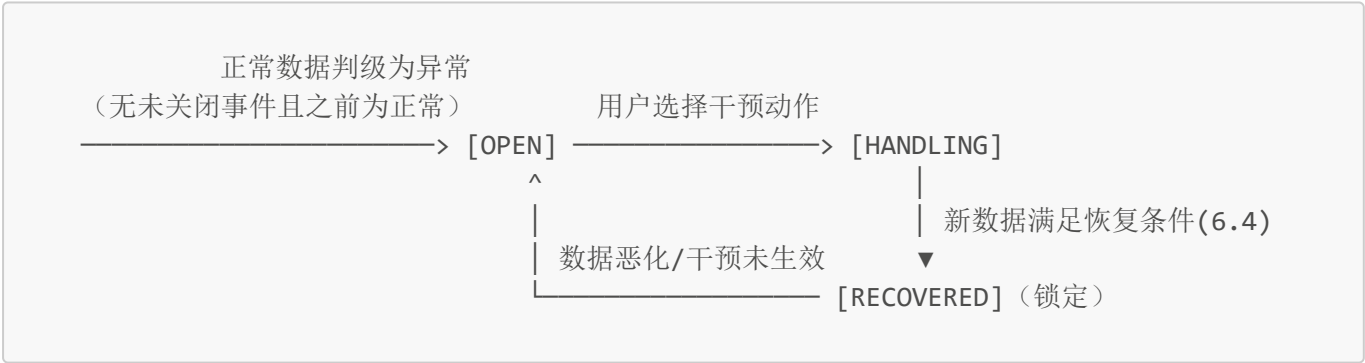
Topic	方向	用途
<code>tidewatch/{reachId}/state</code>	模拟节点 → 三端	断面实时状态
<code>tidewatch/event/update</code>	Web → 移动端/3D	事件创建与状态变化广播（9 字段完整载荷）
<code>tidewatch/intervention</code>	Web ↔ 移动端	干预动作 {event_id, reachId, action}

Web 收到 `tidewatch/+state` 消息后的处理管线（按顺序）：

1. **JSON 解析**：失败即拒绝（D4 故障演示的"发送错误 JSON"触发此层）；
2. **Topic 校验**：`/state` 后缀但断面不在 reach-a/b/c 内 → 拒绝（D4"发送错误 Topic"触发）；
3. **数据质量前置**（contract 8.4 的实现部分）：reachId/waterLevel/turbidity 字段缺失或类型错误、time 缺失/格式错误 → 拒绝并计数；
4. **去重**（contract 6.5）：优先 message_id，否则 reachId+time+载荷哈希；event_id 不参与去重；
5. **乱序/迟到**：time 不晚于该断面最新时间才生效，否则丢弃并计数（迟到消息进 lateMessages 供 F12 查看）；
6. **状态更新**：更新 state、连续异常 streak（正常清零）；
7. **事件状态机**：正常→非正常创建 OPEN；HANDLING 用新数据做恢复判定；
8. **渲染**：卡片、优先关注、事件列表、D5 对照表。

3.3 事件状态机（contract 6.2）

状态：OPEN（待处理）→ HANDLING（处理中）→ RECOVERED（已恢复）。



- 允许的转移：新异常 → OPEN；用户干预 → HANDLING；后续新数据满足恢复条件 → RECOVERED；数据恶化/干预未生效 → 回退 OPEN 或保持。
- 禁止的转移（contract 6.3）：OPEN 直跳 RECOVERED；低置信度感知触发 HANDLING → RECOVERED；迟到/重复消息把已 RECOVERED 改回 HANDLING。
- 恢复判定（contract 6.4）按触发原因：水位异常 → waterLevel < 7.0；水质异常 → turbidity < 60；水流异常 → flowLevel ≤ 1；多原因需全部满足。恢复只能由新数据触发，不能由按钮触发。
- RECOVERED 后事件状态锁定，不再响应后续消息。
- 实现约束：每断面最多一个未关闭事件（openEventByReach 防重复创建）；HANDLING 期间每条新数据追加进 verify_data。

设计权衡：为什么只设 3 个状态、不加 CLOSED——任务书的验收标准只要求 OPEN/HANDLING/RECOVERED 三态（contract 6.2），三态已经完整表达"发现 → 处置 → 恢复"的业务闭环，每个状态都有明确的触发来源：OPEN 由数据触发、HANDLING 由人触发、RECOVERED 由数据触发。加 CLOSED 会引入"人工关闭"这一新触发

器，破坏"恢复必须由数据验证"的设计原则（见 D3 难点），也让状态机的合法转移表成倍膨胀、验证成本上升。RECOVERED 即终态并锁定，配合 6.3 的非法转移清单，使任何消息到达后状态机的行为都是可推导的。

3.4 消息健壮性（contract 6.5）

- **去重**：双键方案——message_id 优先（m: 前缀），否则载荷哈希（reachId|time|waterLevel|turbidity|flowLevel，h: 前缀）；同一条消息只生效一次；
- **乱序**：每断面维护 latestTimeByReach，time ≤ 最新时间的消息不覆盖状态；
- **迟到**：早于当前状态时间戳的消息不回滚已 RECOVERED 事件，最多作历史补充；
- **事件消息同样去重**：event/update 与 intervention 按 message_id 或原始载荷哈希去重，重复广播只生效一次；
- **状态单调守卫**：STATE_ORDER = {OPEN:1, HANDLING:2, RECOVERED:3}，任何消息不得把状态改回更小的值——这是 6.3"非法转移"在跨端场景下的实现方式。

设计权衡：去重为什么用双键——message_id 是 MQTT 消息天然的去重凭证，但本项目的模拟器不携带 message_id（更接近真实水文采集节点的粗放实现），所以需要载荷哈希兜底；哈希取 reachId+time+waterLevel+turbidity+flowLevel 五元组，正是"同一断面同一时刻同一观测"的语义，QoS 1 的重复投递与人为重复发布都会被拦住；event_id 不参与去重是契约明令（6.5），因为同一事件的多条消息（创建、干预、恢复）必须各自生效。乱序为什么用 latestTime 丢弃而不是缓冲重排序——监测场景下"最新即正确"，旧数据对当前状态与事件流转没有价值；缓冲重排需要维护窗口、超时与落盘，复杂度高且可能阻塞实时渲染；直接丢弃 + 计入统计（lateCount）同样满足 6.5 的验收要求，行为可解释。

3.5 感知结果组织（contract 4.2）

E2 的拍照流程生成感知记录并存入页面内存列表：reachId 取当前查看断面（未指定则取优先关注断面），imgId 按 img-{reachId}-{时间戳} 生成，flowLevel 取该断面当前值，confidence=null，source="simulated"，time 为拍照时刻。照片以 dataURL 存内存并显示缩略图，可手动下载为 PNG。感知记录与照片关联当前活跃事件 event_id（如有）。

3.6 Web-移动端同步（E3）

同步由两类消息承载：

- 事件广播：Web 在事件创建、干预、恢复时发布 tidewatch/event/update（完整 9 字段 + source），移动端与 3D 订阅后按 STATE_ORDER 守卫更新；
- 干预广播：两端发布 tidewatch/intervention {event_id, reachId, action}，对端收到后把事件置为 HANDLING 并记录 action；HANDLING 且 action 相同视为重复（幂等），RECOVERED 直接拒绝。

去重与守卫保证：任意一方重复点击、消息重复投递、迟到到达，都不会造成两端状态分叉。

设计权衡：为什么 Web 是事件权威端——Web 监测台是信息密度最高、值班室常开的一端，天然承担"事件台账"职责；移动端可能随时锁屏、断网或关闭页面，若它也能广播事件状态，离线期间的事件进展会丢失，或与 Web 各自演化出分叉状态，对账成本极高。单权威端 + 订阅端单调守卫，让"谁改状态"这个问题只有一个答案，同步逻辑可推导。干预动作则是两端平等：现场巡查员和值班员都可能发起处置，所以 intervention 由两端共同发布、对端响应——权威与平等并存，正好对应"事件是事实、干预是请求"的语义。

4. D1-D5 关键实现

4.1 D1 多断面稳定运行

simulator.py 按"每轮 3 个断面各发 1 条, 共 3 轮, 间隔 3 秒"的节奏发布; status 由 make_status() 按 contract 第 3 节规则计算 (≥ 8.5 洪水预警; ≥ 7.0 高水位警戒; ≥ 60 水质浑浊; 否则正常), 载荷不含任何手填状态。演示数据三断面不串线: reach-a 水位型 (4.5/5.0/7.6)、reach-b 洪水型 (9.0/7.8/6.5)、reach-c 浊度型 (75/68/55)。三端订阅同一 Topic, 一条消息同时驱动三端更新 (Evidence/D1/one_message_three_ends.png、three_ends_online.png)。

难点: 三端串线风险——三个断面共用同一条 wildcard 订阅 (tidewatch/+/state), 任何一端的解析若依赖消息到达顺序或数组下标, 断面的数据就会互相覆盖。**怎么解决:** 所有状态以消息内的 reachId 为唯一 key (contract 第 1 节), 不依赖 topic 顺序; 渲染按固定断面顺序遍历; 模拟器端 topic 与载荷都由 reachId 模板生成, 从源头保证"topic、payload、渲染位置"三者一致。

4.2 D2 持续风险与优先关注

优先分 = 环境分 + 感知分 + 持续分 (contract 7.1): 环境分按状态 3/2/1/0; 感知分直接取 flowLevel (0-3); 持续分按连续异常次数 (≥ 3 次 $\rightarrow$ 2 分, 1-2 次 $\rightarrow$ 1 分, 正常清零)。实现要点: streak 计数挂在每条消息处理之后; 正常消息清零、异常消息 +1; 每次渲染重算所有断面总分, 最高分断面加 **priority** 样式并在顶部横幅显示。优先对象随数据自然切换, 不写死任何断面——Evidence/D2 的三组验证分别展示了 reach-b (洪水+flow2)、reach-a (高水位+flow3)、reach-c (水质浑浊+持续) 各自成为优先关注的场景 (group1.png、group2.png)。

难点: 为什么不用简单的 max(status) 排序——状态等级是定性的: "水质浑浊"与"高水位警戒"哪个更严重? 直接比大小没有公认答案, 而且只看状态会丢失两个重要维度: 水流强度 (flowLevel 0-3) 和异常的持续性 (一次偶然超标与连续一小时超标风险完全不同)。**怎么解决:** 按 contract 7.1 拆成环境分 (规则结论) + 感知分 (flowLevel) + 持续分 (streak 次数) 三个可量化维度加权求和, 量化规则公开、可验证; streak 由每条消息驱动更新, 正常即清零, 保证"持续性"只对连续异常计分。

4.3 D3 事件状态机与消息健壮性

分四步落地: D3-1 正常 $\rightarrow$ 非正常且无未关闭事件时创建 OPEN (problem 按水位/水质/水流多选记录, priority_reason 引用优先分); D3-2 每个 OPEN 事件带"开始干预"按钮, 弹窗三选一 (开闸泄洪/开启泵站/投加净水剂), 点击后 state=HANDLING、action 记录动作, 无动作不变状态; D3-3 恢复判定由后续新数据触发 (6.4), 满足则 RECOVERED 并记录 recover_time, 未满足保持 HANDLING 并追加 verify_data; D3-4 实现 3.4 节的去重/乱序/迟到机制, 并在测试面板提供"发送重复数据""发送旧数据""发送带 message_id 的消息"按钮, 页面顶部显示已处理/重复丢弃/乱序迟到/拒绝计数。反向验证见 Evidence/D3/reverse_verify_recovered_locked.png。

难点: 为什么恢复必须由数据触发——若允许按钮直接把事件改为 RECOVERED, 处置人员可以"一键关闭"事件, 系统就失去了对现场真实恢复的验证能力: 干预是否生效无从得知, 事件台账沦为形式。**怎么解决:** 恢复判定只挂在新数据到达的处理管线里 (6.4), HANDLING 期间每条新数据先追加进 verify_data 再逐条对照恢复条件; RECOVERED 的 recover_time 取真实新数据的 time, 使"已恢复"有客观数据依据; 同时也天然满足 6.3"禁止 OPEN 直跳 RECOVERED"——任何恢复都必须先经过一次人工干预。

4.4 D4 故障演示与修复

按 contract 第 11 节计划制造两类故障: 测试面板"发送错误 Topic"向 tidewatch/wrong-topic/state 发消息 (载荷带 2000 年时间戳, 即使无 Topic 校验的端也会被乱序守卫丢弃, 保证不破坏系统状态); "发送错误 JSON"发布 {reachId: reach-a, waterLevel: }。Web 端解析失败/校验失败后拒绝计数 +1 并显示最近拒绝原因, 三端与事件状态不受影响。修复验证: 点"发送带 message_id 的消息"或跑 simulator, 三端恢复正常更新 (Evidence/D4/before_fault.png、bad_json.png、after_fix.png)。

难点：为什么故障载荷带 2000 年时间戳——故障演示的目的是"展示系统能拒绝错误输入"，而不是"真的搞坏演示环境"。错误 Topic 的消息若只有 Web 端做了 Topic 校验，落到移动端/3D 这类校验较弱的订阅端时仍可能被当正常消息处理，污染状态后还需手动刷新恢复。**怎么解决：**错误 Topic 载荷携带 2000-01-01 00:00:00 时间戳——任何订阅端收到它都会被乱序守卫 ($\text{time} \leq \text{断面更新时间}$) 丢弃，与 Topic 校验构成双保险；演示完故障后无需重启任何一端，发一条正确消息即可恢复。

4.5 D5 规则/ML 对照

python/ml_detect.py 用 pandas 读历史 CSV（自动修复手写段与追加段的行粘连），每个断面独立训练 IsolationForest（特征 waterLevel/turbidity/flowLevel，contamination=0.1，random_state=42），避免跨断面形态互相污染。ML 标签按固定映射从 decision_function 分数与 predict 投票得到：明显异常（投票判异常且分数<0）/ 偏离历史（分数<0）/ 略偏 ($0 \leq \text{分数} < 0.1$) / 接近历史 (≥ 0.1)。每条输出 status_rule（规则重算并与 CSV 交叉核对）、status_ml、is_consistent，写入 web/d5_compare.json。Web 端 fetch 该文件后按 (reachId, waterLevel, turbidity, flowLevel) 四元组为每条实时消息查对照：表格展示规则判断/ML 判断/一致性，不一致行红底；组合裁决按 contract 8.3（规则 0.7 + ML 0.3，规则正常 + ML 明显异常 → 需人工复核）。另有 2 个标注构造样本不参与训练：reach-a (6.8,45,2) 规则正常/ML 明显异常（8.3 需人工复核场景），reach-b (9.2,20,2) 规则洪水预警/ML 略偏（模型把反复出现的历史洪水学成常态）。自然数据同样出现不一致：reach-b/c 的异常形态因反复出现被 ML 判接近历史，reach-a 的 4.7/22 与 5.0/25 被 ML 判明显异常而规则判正常——这正是对照的价值（Evidence/D5/d5_table.png）。

难点：为什么 per-reach 训练而不是全局模型——三个断面的正常形态差异极大：reach-c 常态浊度 70–80 NTU，reach-a 常态只有 20–25 NTU。全局训练会把 reach-c 的常态判成异常（或把阈值拉平后对 reach-a 失敏），ML 对照就退化成规则的影子。**怎么解决：**每个断面独立训练 IsolationForest，特征取 waterLevel/turbidity/flowLevel，"偏离自身历史"成为异常判据，与规则的"绝对阈值"形成真正互补的对照视角；代价是每断面样本量小、分数分布窄（已在第 8 节已知限制说明），但语义正确性优先于样本规模。

5. E1–E3 关键实现

5.1 E1 Three.js 3D 河道场景

map3d/index.html 用本地 three.min.js (r128) + OrbitControls 构建三河段场景：每个断面一个 Group（河床/水面/水位柱/泡沫粒子/标签/事件图标/优先光环）。水位柱高度随 waterLevel 线性映射；水面与柱体颜色随 status（正常绿/高水位橙/洪水红/浑浊紫）；泡沫流动速度随 flowLevel ($0.25 + \text{flow} \times 0.55$)；事件图标按状态闪烁（OPEN 红快闪、HANDLING 橙慢闪、RECOVERED 绿常亮），事件状态来自 event/update 订阅并在无事件流时用本地 OPEN 推断兜底；优先关注断面带金色旋转脉动光环。点击河段用 Raycaster 命中水面/柱体/河床，BoxHelper 高亮描边并显示六字段详情面板。动画循环用独立累加器修复了 getElapsedTime/getDelta 同帧调用导致的冻结问题（Evidence/E1/3d_overview.png、click_detail.png、live_update.png）。

设计权衡：为什么 Three.js 而不是 2D 地图——任务书 E1 的最低完成线就是地图/3D 场景；3D 比 2D 多了一个可用的空间维度：水位柱高度直接承载 waterLevel，水流方向与泡沫动画承载 flowLevel，风险用颜色 + 金色光环 + 事件图标三重编码，巡视者不点选也能一眼定位异常断面；Three.js r128 本地化后零外网依赖，OrbitControls 提供拖拽/缩放交互，代码量可控（单文件约 400 行）即达到 E1 标准。

5.2 E2 Camera / ASR / TTS 多模态交互

Web 页"多模态交互"区三项能力：

- **ASR：**Web Speech API (zh-CN) 识别两条指令——"查看 reach-a/b/c"切换当前查看断面（卡片蓝色描边并滚动定位），"朗读结论"触发 TTS；识别结果实时显示，未识别指令给出提示；浏览器不支持时按钮

给出明确提示；

- **TTS**: SpeechSynthesis 朗读当前查看断面（未指定时取优先关注断面）的真实 MQTT 状态，句式"reach-a, 水位 7.6, 浊度 20, 水流平稳, 状态高水位警戒", 内容全部来自 state 中的实时数据与 flowLevel 映射表, 无写死文本；
- **Camera**: getUserMedia 打开摄像头（优先后置）→ 弹层预览 → 拍照 → canvas 转 dataURL 存内存、显示缩略图、关联 reachId/时间/当前事件 event_id, 并按 3.5 节生成感知记录；"保存图片"下载 PNG。

(Evidence/E2/asr_command.png、tts_speaking.png、camera_photo.png、perception_record.png)

设计权衡: 为什么用浏览器原生 API 而不是第三方服务——Web Speech API

(SpeechRecognition/SpeechSynthesis) 与 getUserMedia 是 Chrome/Edge 内置能力: 零依赖、零密钥、零费用, 单页即可完成 ASR/TTS/拍照; 云端 ASR 服务需要申请密钥、产生网络请求与费用, 且语音数据会外发, 对本阶段的演示需求完全是过度设计。代价是 Firefox 不支持、需用户授权 (已在第 8 节已知限制注明), 换来的是把"语音指令→切换断面→朗读真实数据→拍照生成感知记录"整条链路收敛在一个 index.html 内, 验证与教学都直观。

5.3 E3 事件跨端广播与移动端干预同步

见 3.6 节机制。实现要点: Web 为 event/update 权威端 (移动端永不发 event/update, 杜绝广播环路); 干预消息必须携带 reachId 与 action; 两端共用 STATE_ORDER 单调守卫 + 事件消息去重 + RECOVERED 拒绝 + 同 action 幂等, 保证任意操作顺序下两端状态最终一致 (Evidence/E3/sync_both.png、mobile_to_web.png、recover_sync.png)

设计权衡: 为什么事件广播和干预广播分两个 Topic——两者的语义不同: event/update 是"状态事实", 由权威端 (Web) 单向广播, 订阅方只能接受并按守卫更新; intervention 是"动作请求", 两端平等发起, 对端响应。分成两个 Topic 后, 权限模型一条规则就能说清: 移动端永不发 event/update、两端都可发 intervention, 广播环路从结构上被排除; 同时与 contract 第 5 节的 Topic 定义一一对应, 故障排查时按 Topic 即可定位消息类别。。

6. 测试、故障与验证

6.1 D1-D5 测试

- **D1**: 三端同时打开订阅同一 Topic, 模拟器 3 轮期间三端同步刷新、数据不串线 (Evidence/D1/three_ends_online.png、one_message_three_ends.png);
- **D2**: 构造三组数据让 reach-b / reach-a / reach-c 分别成为优先关注, 验证优先分计算与高亮切换 (Evidence/D2/group1.png、group2.png);
- **D3**: 正向验证 OPEN→HANDLING→RECOVERED 全流程; 反向验证——RECOVERED 后发新异常不回退、重复消息只生效一次、旧消息被丢弃 (Evidence/D3/duplicate_dropped.png、late_message.png、message_id_dedup.png、reverse_verify_recovered_locked.png);
- **D4**: 故障注入 → 观察拒绝统计 → 发正确消息 → 三端恢复 (Evidence/D4/before_fault.png、bad_json.png、after_fix.png);
- **D5**: 对照表逐条核对规则与 ML 输出、不一致行高亮、构造案例人工复核提示; 权重可经 ml_detect.py 或 JS 常量两条路径修改后验证裁决变化 (Evidence/D5/d5_table.png)。

6.2 D4 故障修复流程

按 contract 第 11 节"观察现象 → 定位原因 → 修复 → 重新运行 → 验证恢复"执行: 错误 Topic 消息在 Web 端被 Topic 校验拒绝, 错误 JSON 在解析层被拒绝, 两者都计入"拒绝: N"统计并展示最近拒绝原因; 修复方式即改回

正确 Topic/载荷后重新发布，三端恢复更新。设计上保证故障消息不破坏状态：错误 Topic 载荷带 2000 年时间戳，即使落在无校验的端也会被乱序守卫丢弃。

6.3 D5 规则/ML 对照测试

训练与预测全自动：`python python/ml_detect.py` 一键从 CSV 生成 `web/d5_compare.json`；Web 端实时消息按值查对照。测试覆盖三类结果：一致（规则与 ML 同判）、规则异常而 ML 正常（历史异常被学成常态）、规则正常而 ML 明显异常（触发 8.3 需人工复核）。构造样本保证"值得分析"案例在数据分布变化后依然存在，并逐条标注说明。

6.4 测试矩阵

D1 多断面稳定运行

测试项	输入	预期	实际	证据
三端同时在线	三端打开并订阅 tidewatch/+ /state	三端均显示"已连接"	通过	Evidence/D1/three_ends_online.png
一条消息三端更新	模拟器发布 reach-b 洪水预警	三端同步显示同一值	通过	Evidence/D1/one_message_three_ends.png
断面不串线	3 轮 9 条消息	每断面只更新自己的卡片/河段	通过	three_ends_online.png

D2 持续风险与优先关注

测试项	输入	预期	实际	证据
优先对象切换（组 1）	reach-b 洪水预警 + flowLevel 2	横幅与高亮指向 reach-b	通过	Evidence/D2/group1.png
优先对象切换（组 2）	reach-a 高水位 + flowLevel 3	横幅与高亮切到 reach-a	通过	Evidence/D2/group2.png
第三组（水质浑浊 +持续分）	reach-c 持续异常	优先分随持续分上升	通过	Evidence/D2/README.md 记录

D3 事件状态机与消息健壮性

测试项	输入	预期	实际	证据
OPEN → HANDLING → RECOVERED 正向流程	异常消息 → 干预 → 恢复值消息	三步按 6.2 顺序流转	通过	页面事件列表 + 测试面板操作

测试项	输入	预期	实际	证据
重复消息只生效一次	点"发送重复数据"	重复计数 +1，状态不变	通过	Evidence/D3/duplicate_dropped.png
message_id 去重	点"发送带 message_id 的消息"（同 id 再发）	第二次被丢弃	通过	Evidence/D3/message_id_dedup.png
乱序/迟到消息	点"发送旧数据"	不覆盖新状态，迟到计数 +1	通过	Evidence/D3/late_message.png
RECOVERED 锁定（反向验证）	RECOVERED 后发新异常/旧消息	不回退、不覆盖	通过	Evidence/D3/reverse_verify_recovered_locked.png

D4 故障与修复

测试项	输入	预期	实际	证据
错误 Topic 拒绝	点"发送错误 Topic"	拒绝计数 +1，状态不污染	通过	Evidence/D4/before_fault.png
错误 JSON 拒绝	点"发送错误 JSON"	拒绝计数 +1，显示拒绝原因	通过	Evidence/D4/bad_json.png
修复验证	发正确消息 / 跑 simulator	三端恢复正常更新	通过	Evidence/D4/after_fix.png

D5 规则/ML 对照

测试项	输入	预期	实际	证据
对照数据加载	运行 ml_detect.py + http 打开页面	表格显示规则/ML/一致三列	通过	Evidence/D5/d5_table.png
不一致行红底	模拟器数据（reach-b 洪水预警 vs ML 略偏）	该行红底 + "不一致"	通过	Evidence/D5/d5_table.png
8.3 需人工复核	手动发布 reach-a (6.8, 45, 2)	裁决显示"需人工复核"	通过	Evidence/D5/d5_table.png

7. 自主设计与开放拓展

7.1 自主设计点

1. **前端库全面本地化**：诊断出 unpkg CDN 在目标网络不可达导致"未连接"后，把 mqtt.js / three.js / OrbitControls 全部下载到仓库，并在 mqtt.connect() 前加 `typeof mqtt === "undefined"` 守卫（红色提示 + throw），彻底摆脱外网依赖；
2. **消息健壮性测试面板**：把 D3-4 的去重/乱序/迟到做成可点击按钮（重复/旧数据/message_id/错误 Topic/错误 JSON），并新增"手动发布"表单（reachId + 三数值输入，status 与 time 自动生成），使状态机全流程可在页面上徒手复现；
3. **故障安全设计**：D4 故障载荷带 2000 年时间戳，保证即使某端缺少 Topic 校验也不会被污染；
4. **CSV 自动修复**：ml_detect.py / analyze.py 读取时自动修复手写历史末尾无换行造成的行粘连，不改动原文件；
5. **D5 双路径权重验证**：权重既由 ml_detect.py 写入 JSON 展示，也以 JS 常量参与裁决计算，两条修改路径都可验证权重对裁决的影响。

7.2 开放拓展（contract 第 12 节，本阶段未做）

真实视觉感知（source=model，flowLevel 带置信度）、FastAPI/SQLite 持久化后端（事件与照片落库、跨刷新恢复）、PWA 移动端、云端部署、更多故障演练类型（contract 11 未勾选项：停止 Broker、改错字段名、断面长时间不发数据、感知服务失败）。

8. 已知限制

- flowLevel 本阶段来源为 simulated（contract 4.3），无真实视觉模型，感知记录 confidence 恒为 null，不伪造置信度；
- 固定规则阈值仅用于编程练习，不代表真实水文防汛标准（contract 13）；
- IsolationForest 训练样本量小（每断面数十条），标签阈值是固定定义；历史累积后重跑 ml_detect.py 可更新对照，但误报/漏报风险存在；
- 事件、去重集合、照片、感知记录均为页面内存态，刷新即重置，无后端持久化；
- D5 对照区依赖 fetch，页面必须经 http 访问（file:// 直开会被 Chrome 拦截）；ASR/TTS 仅 Chrome/Edge 支持；
- 移动端按巡查分工只显示优先关注断面，不显示全部断面列表；
- 系统面向本机/局域网演示，未做云端部署（contract 13）。